

Análisis de datos

Carrera de Especialización en
Inteligencia Artificial



Programa de la materia

1 **Introducción al análisis de datos**
Flujo de trabajo típico. Aplicaciones. Configuración del entorno. Introducción a las principales bibliotecas.
28/8

2 **Análisis exploratorio de datos (EDA)**
Tipos de variables. EDA y visualización de variables numéricas y categóricas. Identificación de valores faltantes y outliers. Relación entre variables.
4/9

3 **Taller práctico - parte 1**
Trabajo en grupos: EDA y visualización inicial de un dataset. Buenas prácticas. Puesta en común.
11/9

4 **Preprocesamiento y limpieza de datos**
Flujo de preparación de datos. Prevención de data leakage. Tratamiento de datos faltantes y outliers. Cardinalidad. Codificación. Discretización.
18/9

5 **Procesamiento (continuación)**
Normalización y estandarización. Técnicas avanzadas para el tratamiento de datos faltantes y outliers. Desbalance. Creación de nuevos features.
25/9

6 **Reducción de dimensionalidad**
Métodos de extracción y técnicas para selección de features. **Bonus:** EDA de datos no estructurados.
2/10

7 **Taller práctico - parte 2**
Trabajo en grupo: preprocesamiento y feature engineering de un dataset. Buenas prácticas. Puesta en común. Consultas.
9/10



13/10 - límite para compartir el material para la exposición final.

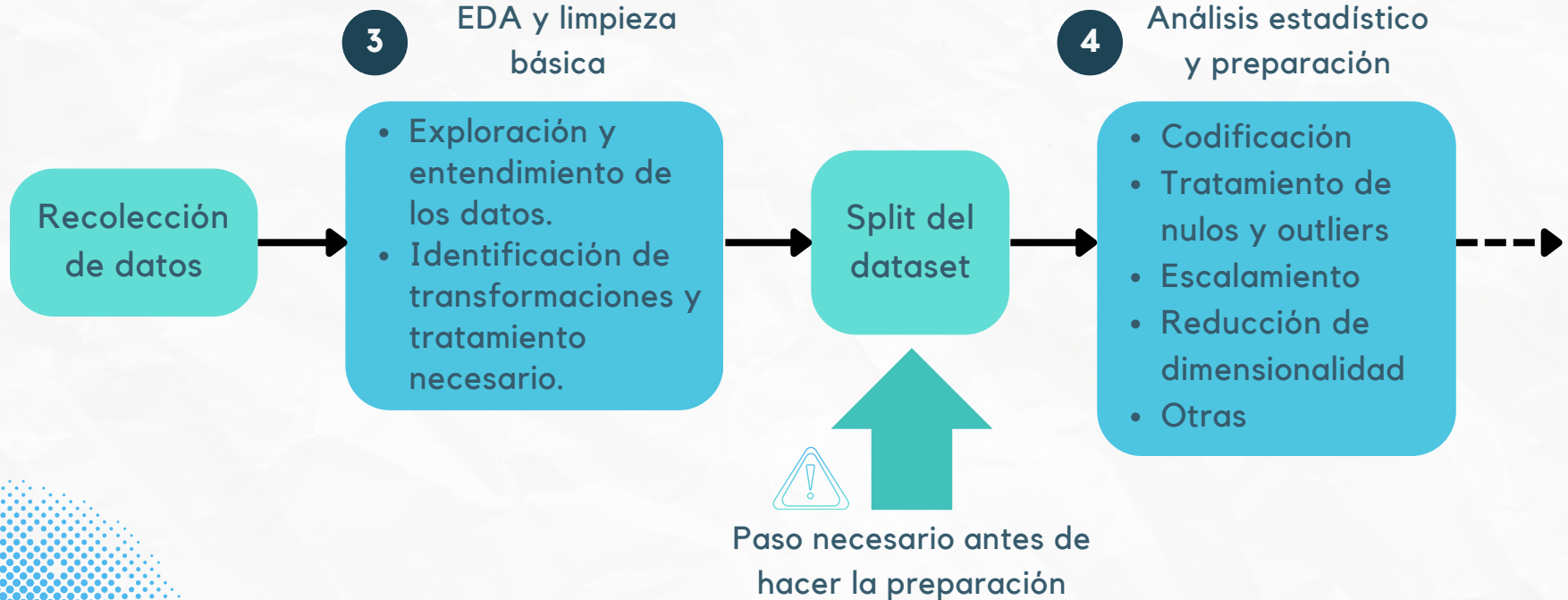
8 **Exposición final**
Exposición de trabajos finales.
16/10

Bonus: Herramientas para automatizar el EDA y otras tareas del flujo de trabajo.

Flujo de trabajo del análisis de datos para IA



Flujo de preparación de datos para IA



Es buena practica primero hacer el split antes del paso 4, esto es para evitar el error de Data Leakage

➔ Data leakage

Es un problema que sucede cuando cierta información que no se suponía que el modelo debía conocer, se filtra al momento del entrenamiento.



Como resultado, al evaluar el modelo se obtienen métricas de desempeño engañosas (demasiado optimistas) y mala performance con datos nuevos.

El data leakage se da porque al trabajar con TODO el dataset se computan características de test en train.

Es decir, se va a entrenar "conociendo" cosas de test, por lo tanto al evaluar el modelo va a performar mejor de lo que debería. Pero, en producción el modelo puede no generalizar!!!

La diferencia con el overfitting, donde se evalúa con datos de train, el data leakage se presenta con datos en producción.

Tipos de data leakage

- Target leakage: se entrena el modelo con datos que no estarían disponibles en un escenario real.
- Contaminación train-test: se entrena el modelo con información de los datos de test (usualmente ocurre al preprocesar los datos).



Ejemplo: target leakage

Una empresa de celulares quiere usar IA para predecir cuántos clientes se perderán este año.

Para entrenar el modelo, el equipo utiliza las siguientes variables:

`id_cliente`, `fecha_alta`,
`servicio contratado`, `deuda`,
`nro_reclamos` y `fecha_baja`.

Esta contaminación se puede evitar mejor al hacer feature engineering



Sería un target leakage porque no es un dato conocido.
Se estaría entrenando el modelo con un dato que no tengo en el futuro.

`fecha_baja` sería casi idéntico al target

Ejemplo: contaminación train-test

Un profesor está entrenando a un grupo de alumnos en estadística y quiere evaluar cuánto aprendieron.

En el examen, les toma 5 ejercicios pero 3 de ellos son exactamente iguales a los vistos en la práctica.

Se usa ejercicios que el alumno ya estudio. No se puede determinar que realmente sabe.

Siempre se quiere que en test, el modelo pruebe datos que nunca vio



Esta contaminación se evita al hacer el split de train y test

Trucos para evitar data leakage

- Prevenir target leakage:
 - Analizar los features disponibles. Evitar incluir features que conducen directamente a la variable target o que no estarían disponibles en un escenario real.
 - Cuidado al dividir en train y test, por ejemplo, no incluir datos futuros. Tener cuidado con los duplicados!
- Prevenir contaminación train-test:
 - Realizar el split antes de cualquier preprocesamiento. Calcular las transformaciones sobre los datos de entrenamiento únicamente, y luego aplicarlas a los datos de test.



Consideraciones al hacer el split

x1	x2	target
01/2024		
...		
...		
04/2024		
...		
...		
09/2024		
10/2024		
...		
12/2024		

SERIE TEMPORAL

OJO con los datos temporales! Se debe separar por orden temporal (mantener un orden cronológico)

x1	x2	target
CLIENTE 1		
CLIENTE 1		
CLIENTE 1		
CLIENTE 2		
CLIENTE 2		
CLIENTE 3		
CLIENTE 3		
CLIENTE 4		
CLIENTE 4		
CLIENTE 5		

CLUSTERS DE DATOS

(El dataset contiene múltiples muestras de la misma entidad: clientes, pacientes, etc.)

TRAIN

TEST

No es conveniente en test mezclar datos de distintos grupos.

Por ejemplo, si en train y test hay datos del cliente_1, cuando el modelo se entrene, aprende del cliente_1 y en test ya lo conoce. Se introduce data leakage.

Es conveniente que la data de un mismo cliente este en train o test, no en ambos.

→ Estrategias para manejo de datos faltantes y outliers

Realizado el split de los datos, vamos a trabajar con los datos de entrenamiento

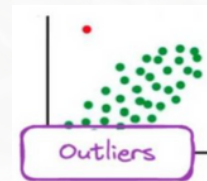
¿qué son los datos faltantes?

	F	G	H	I	J
A	0.620576	0.140053	1.352728	NaN	0.808078
B	NaN	0.526829	NaN	NaN	0.170902
C	NaN	0.458827	1.406713	0.071119	NaN
D	NaN	2.307197	NaN	NaN	NaN
E	0.203402	0.259913	NaN	0.505811	1.516755

Missing data

Son observaciones incompletas o valores no registrados en el dataset (NaN, null).

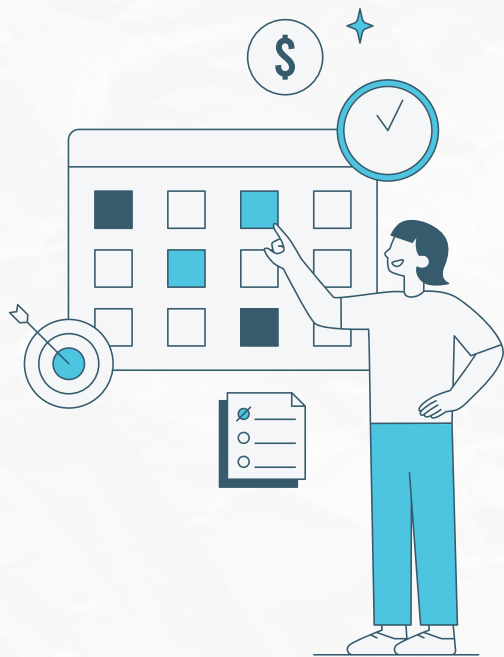
¿qué son los outliers?



Son puntos que difieren significativamente de otras observaciones en un dataset.

Si no se manejan adecuadamente, los datos faltantes y los outliers pueden distorsionar el análisis, introducir sesgo en los modelos y conducir a conclusiones erradas.

Estrategias para manejo de datos faltantes



Eliminación:

- Borrar filas (observaciones).
- Borrar columnas (variables).

Solo es conveniente cuando casi toda la fila o columna es nula.

Imputación (colocar un valor):

- Métodos univariados (se usan los datos de la variable con faltantes).
 - Por media, mediana, moda.
 - Valor de adelante o de atrás.
 - Crear nueva categoría.
 - Series temporales: hacia adelante/atrás e interpolación lineal o polinómica.

En la imputacion, los datos son siempre "inventados". Puede estar bien o no, hay que tener cuidado.

Ejemplo: imputación por la mediana

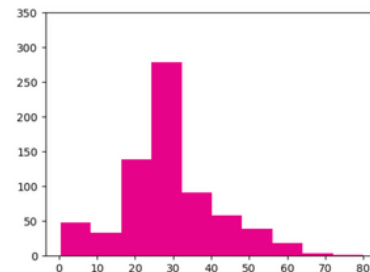
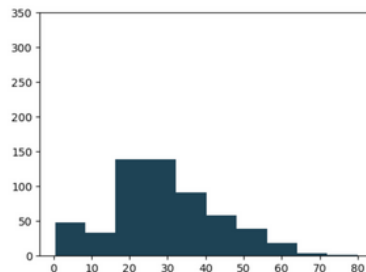
columna original

Imputación

Mediana de la
edad = 28

sex	fare	class	deck	embark_town	alive	alone	age	age_2
male	26.5500	First	E	Southampton	yes	True	51.00	51.00
female	21.0750	Third	NaN	Southampton	no	False	3.00	3.00
female	19.2583	Third	NaN	Cherbourg	yes	False	0.75	0.75
male	7.2292	Third	NaN	Cherbourg	no	True	NaN	28.00
male	8.0500	Third	NaN	Southampton	no	True	34.00	34.00
female	23.0000	Second	NaN	Southampton	yes	False	18.00	18.00
female	27.7208	First	B	Cherbourg	yes	True	44.00	44.00
male	14.5000	Third	NaN	Southampton	no	True	NaN	28.00
male	9.5000	Third	NaN	Southampton	no	True	NaN	28.00
female	9.5875	Third	NaN	Southampton	no	True	37.00	37.00
male	11.5000	Second	NaN	Southampton	no	False	21.00	21.00
male	78.8500	First	C	Southampton	no	False	36.00	36.00

Histograma de la edad antes y
después de la imputación



Para la categoria, se puede usar la moda para los datos faltantes

Ejemplo: imputación hacia adelante/atrás

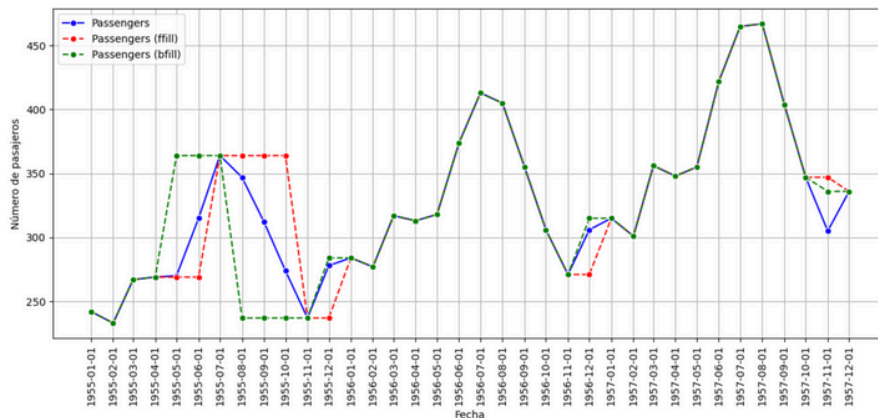
columna original

Imputación

	date	passengers	pass_ffill	pass_bfill
0	1949-01-01	NaN	NaN	118.0
1	1949-02-01	118.0	118.0	118.0
2	1949-03-01	132.0	132.0	132.0
3	1949-04-01	129.0	129.0	129.0
4	1949-05-01	NaN	129.0	135.0
5	1949-06-01	135.0	135.0	135.0
6	1949-07-01	148.0	148.0	148.0
7	1949-08-01	148.0	148.0	148.0
8	1949-09-01	136.0	136.0	136.0
9	1949-10-01	119.0	119.0	119.0
10	1949-11-01	NaN	119.0	126.0
11	1949-12-01	NaN	119.0	126.0
12	1950-01-01	NaN	119.0	126.0
13	1950-02-01	126.0	126.0	126.0

Otra opción sería tomar el promedio del dato anterior y posterior.
Un método un poco más avanzado, es hacer una interpolación lineal.

Número de pasajeros antes y después de la imputación



¿Cómo elegir la estrategia para el tratamiento de faltantes? 1/2

- Si los faltantes son MCAR:
 - Eliminación de fila cuando los datos faltantes representan 5-10% máx. del total de filas.
 - Eliminación de columna: >60-70% valores faltantes en la misma columna y la variable no es importante.
 - Atención: evaluar el impacto en la distribución de los datos antes de eliminar filas. Eliminar datos puede producir sesgo.
- Si son MCAR o MAR y el porcentaje de faltantes es bajo (10-30%):
 - Imputación por media: datos con distribución aprox. Normal.
 - Imputación por mediana (captura mejor relaciones no lineales pero depende de la calidad de otras variables y puede implicar más costo computacional).
 - Imputación por moda (variables categóricas) o nueva categoría (ej., "FALTANTE") si tiene sentido.
 - Atención: imputar con la media, mediana y moda modifican la varianza.

Cuando los faltantes son MCAR, al eliminar el dataset no queda sesgado.

Pero si son MAR, al eliminar si puede quedar sesgado. Es menos recomendable eliminar cuando es MAR.

¿Cómo elegir la estrategia para el tratamiento de faltantes? 2/2

- Si son MAR con patrones claros y el porcentaje es moderado a alto (20-50%)
 - Técnicas avanzadas (las veremos más adelante)
- Si se sospecha de MNAR o MAR complejos (dependen de muchas variables):
 - Técnicas avanzadas (las veremos más adelante)
- Series temporales:
 - se elige una imputación específica según el tipo de serie (interpolación, modelos de series temporales como ARIMA, LSTMs, modelos generativos).

Estrategias para manejo de outliers

- **Eliminación.**
 - Si no son representativos o se los considera errores.
 - **Imputación.**
 - Reemplazar por un valor acorde (media, mediana).
 - Aplicar winsorización (o *capping* en inglés): reemplazar por un valor máximo o mínimo definido, generalmente basado en un percentil.
 - **Transformación** (Para minimizar el impacto)
 - Logarítmica (o raíz cuadrada)
 - **Segmentación.**
 - Agrupar los outliers en una categoría separada (variables categóricas)
- Analizar si se trata de errores o son valores legítimos!

Ejemplos prácticos en Jupyter



Cardinalidad (v. categóricas)

Es el número de valores únicos que puede tomar una variable en un conjunto de datos.

Baja

La mayoría de los valores se repiten



Estado civil

Día de la semana

Alta

Muchos valores únicos



Nombres de clientes

Direcciones de correo electrónico

Extrema

Cada valor es único en la mayoría de los casos.



Números de serie

Códigos de transacciones

Codificación

- Consiste en transformar **variables categóricas** en **números** para que puedan ser utilizadas por algoritmos de ML.
- La elección de la estrategia de codificación depende fuertemente de la **cardinalidad** de los datos.



La mayoría de los modelos necesitan que sus variables sean numericas.

Estrategias de codificación

También se conoce como variables dummies

One-hot

...	COLOR	...	PRECIO
...	AZUL	...	8
...	ROJO	...	10
...	VERDE	...	5
...	AZUL	...	16
...	VERDE	...	7
...	VERDE	...	3

AZUL	ROJO	VERDE
1	0	0
0	1	0
0	0	1
1	0	0
0	0	1
0	0	1

Label

COLOR_L
10
20
30
10
30
30

Ordinal

COLOR_O
0
1
2
0
2
2

Util cuando el color es la variable target. Para una feature, el modelo puede interpretar un orden

Util si el color tiene un peso mayor respecto a otro

Estrategias de codificación

¿Estamos ante un caso de data leakage?

Sí, al usar cierto valor relacionado con el target, es como un data leakage.



...	COLOR	...	PRECIO
...	AZUL	...	8
...	ROJO	...	10
...	VERDE	...	5
...	AZUL	...	16
...	VERDE	...	7
...	VERDE	...	3

Target

COLOR_T
12
10
5
12
5
5

$$\text{Azul} = (8 + 16) / 2 = 12$$

$$\text{Rojo} = (10) / 1 = 10$$

$$\text{Verde} = (5 + 7 + 3) / 3 = 5$$

Este encoding conserva la relación que tiene el feature con el target.

Una mejora a este metodo, es usar k-fold

Estrategias de codificación

¿Estamos ante un caso de data leakage?



...	COLOR	...	PRECIO
...	AZUL	...	8
...	ROJO	...	10
...	VERDE	...	5
...	AZUL	...	16
...	VERDE	...	7
...	VERDE	...	3

Target

COLOR_T
12
10
5
12
5
5

Para reducir el data leakage se puede implementar una variante:

k-fold target encoding

o

leave-one-out target encoding

[Video](#)

Estrategias de codificación

...	COLOR	...	PRECIO
...	AZUL	...	8
...	ROJO	...	10
...	VERDE	...	5
...	AZUL	...	16
...	VERDE	...	7
...	VERDE	...	3

Target	<u>Frequency</u>
COLOR_T	COLOR_F
12	0.333
10	0.166
5	0.5
12	0.333
5	0.5
5	0.5

El inconveniente es que existan valores con la misma frecuencia

$$\text{Azul} = 2/6 = 0.333$$

$$\text{Rojo} = 1/6 = 0.166$$

$$\text{Verde} = 3/6 = 0.5$$

Estrategias de codificación

...	COLOR	...	PRECIO
...	AZUL	...	8
...	ROJO	...	10
...	VERDE	...	5
...	AZUL	...	16
...	VERDE	...	7
...	VERDE	...	3

Target	Frequency	<u>Hashing</u>	
COLOR_T	COLOR_F	H1	H2
12	0.333	0	1
10	0.166	1	0
5	0.5	1	1
12	0.333	0	1
5	0.5	1	1
5	0.5	1	1

Hashing codifica una categoría a código binario. El inconveniente es que puede existir colisiones si no se elige la cantidad de bits conveniente.

Es útil cuando se tiene una cardinalidad muy alta

Hashing encoding: aclaraciones

- En general es la solución preferida cuando se tienen variables categóricas con cardinalidad muy alta y se trabaja con pipelines simples y modelos de ML tradicional.
- Los enfoques híbridos con embeddings* pre-entrenados son una alternativa, pero implican mayor complejidad. Por lo general se usan para capturar semejanzas semánticas entre categorías.

El inconveniente de hashing es que no mantiene ninguna relación. Como vemos en el ejemplo, ML Engineer y Senior Developer están relacionados, no así Analista de RRHH.

ID_EMPL	TITULO	SUELDO
E00001	MACHINE LEARNING ENGINEER	...
E00002	SENIOR SOFTWARE DEVELOPER	...
E00003	ANALISTA DE RECURSOS HUMANOS	...
...	...	...

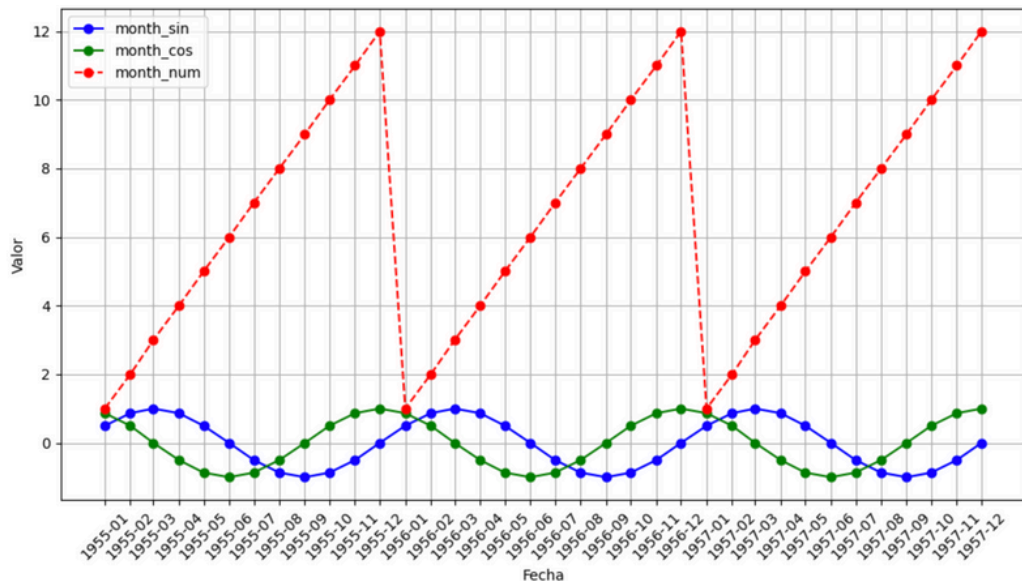
Los embeddings permiten mantener cierta relación semántica.

(*) Embeddings se explica en detalle en la materia Aprendizaje Profundo.

Estrategias de codificación

date	month	month_num	month_sin	month_cos
1949-01-01	Jan	1	0.5000	0.8660
1949-02-01	Feb	2	0.8660	0.5000
1949-03-01	Mar	3	1.0000	0.0000
1949-04-01	Apr	4	0.8660	-0.5000
1949-05-01	May	5	0.5000	-0.8660
1949-06-01	Jun	6	0.0000	-1.0000
1949-07-01	Jul	7	-0.5000	-0.8660
1949-08-01	Aug	8	-0.8660	-0.5000
1949-09-01	Sep	9	-1.0000	-0.0000
1949-10-01	Oct	10	-0.8660	0.5000
1949-11-01	Nov	11	-0.5000	0.8660
1949-12-01	Dec	12	-0.0000	1.0000
1950-01-01	Jan	1	0.5000	0.8660
1950-02-01	Feb	2	0.8660	0.5000
1950-03-01	Mar	3	1.0000	0.0000

Cyclic encoding



En el grafico se ve que si se usa el numero del mes, el salto entre dic y ene es muy grande.

Esta codificación, es útil cuando se tiene cierto patrón en el tiempo

Para variables binarias, se codifica con 0/1 o tambien se puede hacer one-hot encoding

Ejemplos prácticos en Jupyter



RESUMEN de estrategias de codificación

Técnica	Consiste en ...	Ideal para...
One-Hot Encoding	crear una columna binaria (0/1) para cada categoría.	<u>Baja cardinalidad</u> . Multi-label (multi-hot encoding)
Label Encoding	asignar un número entero arbitrario a cada categoría	Codificar la <u>variable target</u>
Ordinal Encoding	asignar un número entero secuencial a cada categoría.	Variables con un <u>orden lógico</u>
Target Encoding	reemplazar cada categoría con la media (u otra estadística) de la variable objetivo.	<u>Alta cardinalidad</u> . Problemas supervisados
Frequency Encoding	sustituir cada categoría por su frecuencia de aparición.	<u>Muchas categorías frecuentes</u> (si hay categorías poco frecuentes, agrupar). Hay relación fuerte entre la frecuencia de una categoría y la variable target (ej. fraude)
Hashing Encoding	aplicar una función hash para mapear categorías en un número fijo de dimensiones.	<u>Cardinalidad extrema</u>
Cyclic Encoding	transformar variables cíclicas en coordenadas en un círculo mediante funciones seno y coseno.	Variables con <u>patrón cíclico</u>

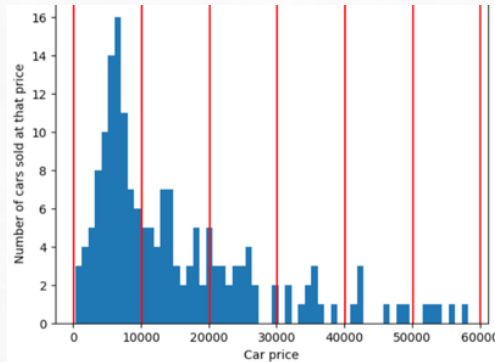
Estrategias de codificación. Ventajas y desventajas.

Técnica	Ventajas	Desventajas
One-Hot Encoding	Simple y efectiva	Escala mal con alta cardinalidad
Label encoding	Simple y efectiva	Pensado únicamente para la variable target (porque implica orden)
Ordinal Encoding	Conserva el orden natural	Crea relaciones numéricas artificiales, (lineal, igualmente espaciada, por ej: low=1, medium=2, high=3) y puede confundir a los modelos lineales o basados en distancia
Target Encoding	Compacto y efectivo	Riesgo de sobreajuste
Frequency Encoding	Útil para reducir categorías raras	Puede perder información
Hashing Encoding	Escalable y eficiente	Puede provocar colisiones (no es reversible) No conserva relaciones semánticas, orden ni jerarquía
Cyclic Encoding	Conserva continuidad temporal	No se recomienda en datos sin periodicidad clara

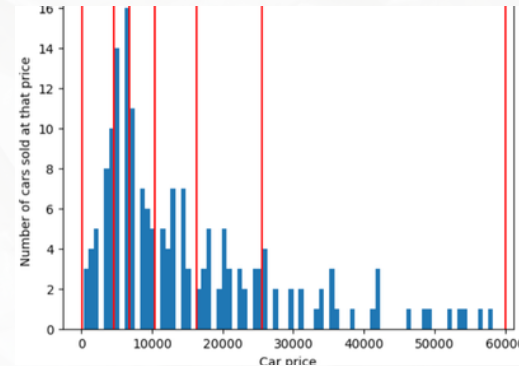
Discretización

- La discretización convierte **variables continuas** en variables discretas o **categorías**.
- Se usa para mejorar la interpretación o el rendimiento de un modelo.

Métodos de discretización



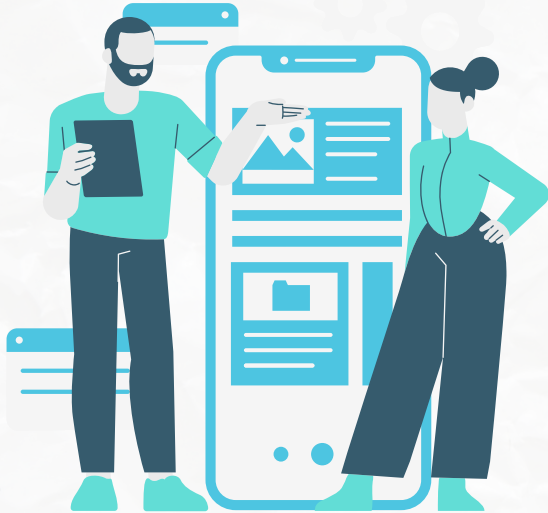
- Por intervalos de igual rango de valores.



- Por puntos de corte (cuantiles)
Con cuantiles, cada categoría tiene la misma cantidad de datos

¿Por qué discretizamos?

Cuando el target es continua y no interesa un valor puntual, conviene discretizarla.



- Para reducir la cantidad de datos y la complejidad computacional.
- Con el objetivo de disminuir el "ruido": muchas veces los datos crudos incluyen fluctuaciones aleatorias o irrelevantes que contribuyen al overfitting.
- Cuando se requiere convertir una variable continua en categórica (ej: adaptar la variable target para clasificación).
- Para mejorar el entendimiento o la interpretación de un problema (por ejemplo, convertir edades de clientes en segmentos objetivo: joven, adulto, adulto mayor).
Para el caso de precios: bajo, medio, alto
- Como herramienta para "contener" el efecto de los outliers.
Ahora los outliers tienen un valor, ejemplo "muy alto"

Ejemplos prácticos en Jupyter



RESUMEN de métodos de discretización

Método	Descripción	Pros y contras
Intervalos iguales	Rangos de igual amplitud.	Simple. <u>No sirve si los datos son asimétricos.</u>
por puntos de corte	Basados en la distribución de los datos. Cada intervalo tiene el mismo número de observaciones.	Funciona bien para datos <u>asimétricos.</u>
K-means	Agrupar los datos en clusters con el algoritmo k-means	Captura agrupamientos naturales. Para datasets grandes <u>requiere mucho cómputo.</u>
Ancho fijo	Definidos por el usuario basados en el conocimiento del dominio.	Control total, pero <u>requiere conocimiento de los datos y su distribución.</u>

Es un método de clustering no supervisado

Recap:

* Clase 2: Se utilizaron tecnicas de visualizacion para ver como son las variables y si tienen nulos, outliers

* Clase 4: Se empieza a arreglar los problemas

-> split del dataset para evitar el data leakage

-> tratamiento de nulos y outliers

-> codificación de variables categóricas

-> discretización de variables continuas



¿Te quedaron dudas?

- Recordá que podés consultarnos durante la clase preniendo la cámara y hablando, o simplemente escribiendo en el chat.
- También podés escribirnos por correo las veces que sea necesario.



Esp. Lic. María Carina Roldán
macroldan@fi.uba.ar



Esp. Ing. Ariadna Garmendia
arigarmendia@gmail.com

Y si tenés un feedback, un pedido, una sugerencia, etc. y no te animás a decirlo, podés dejarlo por escrito en la encuesta voluntaria y anónima (disponible durante todo el bimestre).